ECORIDE

Fleet Platform

ARCHITECTURE STRATEGY

SPEC-1-ECORIDE-Architecture-Strategy

Version	1.0
Status	Draft
Author	Andrew Borresen (Product Architect)
Last Updated	2026-04-17

Document Control

Document Title	ECORIDE Architecture Strategy
Version	1.0
Status	Draft
Author	Andrew Borresen (Product Architect)
Last Updated	2026-04-17

Revision History

Version	Date	Author	Change Description
1.0	2026-04-17	Andrew Borresen (Product Architect)	Initial version
1,1	2026-05-05	Andrew Borresen (Product Architect)	Minor clarifications

Sign-Off

Role	Name	Signature	Date
Product Architect	Andrew Borresen		
Lead Developer	Mario Faure		
Product Owner	Lennie De Villiers		

Table of Contents

Contents

Document Control 2
Revision History 2
Sign-Off 2
Table of Contents 3
1. Background 4
2. Requirements 5
3. Method 6
3.1 Tenant Hierarchy & Operating Model 6
3.2 Architecture Overview (Simple View) 7
3.3 Core Architectural Principles (NON-NEGOTIABLE) 7
3.4 Global Platform Readiness (NON-NEGOTIABLE) 7
3.5 Core Platform Capabilities 9
3.6 Modules Overview 9
3.7 Architecture Evolution 9
3.7 Architecture Evolution (by component) 10
3.8 Data Architecture Evolution 10
3.9 Executive Architecture View (Simplified) 11
4. Implementation 12
Guidelines 12
5. Milestones 12
6. Gathering Results (KPIs) 13

1. Background

ECORIDE Technologies is building a real-time fleet operating platform supporting both electric (EcoRide) and LPG (Khula Rides) vehicle fleets.

The platform is not a traditional fleet system. It is a real-time operational control system designed to:

- Maximise vehicle utilisation (24/7)
- Enforce driver discipline and performance
- Provide full operational visibility
- Support multi-fleet, multi-region operations

The architecture evolves pragmatically:

- MVP — rapid delivery (monolithic). Optimised for speed and simplicity
- Production — modular and scalable. Focus on scalability and separation
- Scale — distributed, enterprise-grade. Designed for resilience and high throughput

2. Requirements

Must Have
✓ Multi-tenant model (Account → Fleet → Vehicle)
✓ Mobile applications must run at native (offline) applications IOS and Android (apps downloadable from stores)
✓ Multi-currency, multi-territory, multi-timezone support
✓ Real-time telemetry ingestion (1–5 seconds per vehicle)
✓ Driver performance and enforcement model
✓ Financial flows and reconciliation
✓ High availability and scalability

Should Have
◎ Event-driven architecture
◎ Observability (logs, metrics, tracing)
◎ Modular service boundaries

Could Have
◇ AI-driven optimisation (post-MVP)

3. Method

3.1 Tenant Hierarchy & Operating Model

Ownership Structure

```
ACCOUNT (Business / Customer)
 |
 ├──── FLEET
 |     |
 |     ├──── VEHICLE
 |     ├──── VEHICLE
 |     └──── VEHICLE
 |
 ├──── FLEET
 |     |
 |     ├──── VEHICLE
 |     └──── VEHICLE
 |
 └──── DRIVERS (Assigned to Fleets)
       |
       ├──── Driver A
       ├──── Driver B
       └──── Driver C
```

Operational Assignment (Shift Layer)

```
SHIFT
 |
 ├──── Driver A ──────────▶ Vehicle 1
 ├──── Driver B ──────────▶ Vehicle 2
 └──── Driver C ──────────▶ Vehicle 3
       (During Shift)
```

Key Principle

Drivers are not part of the ownership hierarchy. They are operational actors assigned dynamically to vehicles via shifts.

3.2 Architecture Overview (Simple View)

The ECORIDE platform acts as a central hub connecting:

Users	External Systems
• Drivers • Fleet Managers • Administrators	• Vehicles (via telematics devices) • Ride-hailing platforms • Payment and banking systems • Notification services • AI platform (future)

How the platform operates:

- Collects real-time data
- Organises data by Account → Fleet → Vehicle
- Applies rules and performance scoring
- Dynamically assigns drivers via shifts

3.3 Core Architectural Principles (NON-NEGOTIABLE)

Principle	Details
Tenant Isolation	Every domain entity MUST include account_id and fleet_id. No cross-tenant access.
API-First & Stateless	All functionality exposed via APIs. No server-side session state.
Real-Time & Event-Driven	Platform operates in near real-time. Events emitted for all key actions.
Rules-Driven	Business rules must be configurable. No hardcoded operational logic.
Security & Governance	Encryption in transit and at rest. RBAC + ABAC access control. Full audit logging.

3.4 Global Platform Readiness (NON-NEGOTIABLE)

Area	Requirements
Multi-Currency	• Store original amount (minor units), currency code (ISO 4217), and normalised base value • No implicit conversions — exchange rates must be auditable

Area	Requirements
Multi-Timezone	• **Store all timestamps in UTC** • Apply timezone at presentation layer • Account, Fleet, and Users must have timezone context
Multi-Territory	• Data residency configurable • Support POPIA and GDPR • Regional compliance enforced
Data Privacy & Security	• Encryption in transit and at rest • Controlled access (Row-Based Access Control + Activity-Based Access Control) • Full audit trails

3.5 Core Platform Capabilities

Capability	Description
Driver Performance Engine	Aggregates telemetry, trip, and attendance data. Produces driver scores and tiers.
Rules & Enforcement Engine	Applies business rules. Triggers warnings, restrictions, and suspensions.
Shift Optimisation System	Dynamically allocates drivers. Maximises vehicle utilisation.
Real-Time Processing Layer	Processes telemetry and operational events. Enables low-latency decisions.
Financial & Reconciliation Engine	Handles payouts and reconciliation (business versus personal trips). Supports cash and digital flows.

3.6 Modules Overview

Module	Module
Driver Management	Cash Management
Driver Performance	Fuel / EV Optimisation
Shift Management	Operations Dashboard
Vehicle Management	Admin Dashboard
Fleet Maintenance & Lifecycle	Compliance & Violations

3.7 Architecture Evolution

Stage	Model	Key Characteristics	Trigger
MVP	Modular Monolith	• Fast delivery • Single deployable • Internal modular design	Scaling limits
Production	Modular Services	• Partial separation • Independent scaling • Improved resilience	Increased load
Scale	Microservices	• Fully decoupled	High complexity

Stage	Model	Key Characteristics	Trigger
		• Domain-driven • Independent deployment	

3.7 Architecture Evolution (by component)

Component	MVP	Production	Scale
Front-End	• Web app (React) • Hosted on Replit / Vercel	• Web + Mobile (React / React Native) • CDN (CloudFront / Cloudflare)	• Multi-channel (Web, Mobile, Partner APIs) • Edge delivery (CDN + edge compute)
Back-End	• Node.js modular monolith • Express / NestJS	• Modular services (Node.js / NestJS) • Containerised (Docker)	• Microservices • Kubernetes (EKS) / serverless (Lambda)
Database (Transactional)	• PostgreSQL (single instance)	• Managed DB (AWS RDS PostgreSQL) • Read replicas	• Aurora PostgreSQL / sharded PostgreSQL • Hybrid tenant isolation
Real-Time / Events	• Basic polling • REST APIs	• WebSockets (Socket.io / API Gateway WS) • Event bus (SNS/SQS)	• Streaming platform (Kafka / MSK / Kinesis) • Event-driven architecture
Telemetry / IoT	• Direct ingestion (REST)	• Ingestion service (API Gateway + Lambda)	• IoT platform (AWS IoT Core) • Stream processing (Kinesis / Kafka)
Tenancy Model	• Shared DB (Account_ID and fleet_id isolation)	• Optimised shared tenancy • Logical partitioning	• Hybrid model • Dedicated DBs for premium tenants
Infrastructure	• Replit / simple hosting	• AWS (EC2 / ECS) •Load balancer (ALB)	• Multi-region AWS • Auto-scaling + failover
Observability	• Basic logging	• Centralised logging (CloudWatch / ELK)	• Full observability (Prometheus + Grafana + tracing)

*Technology choices are indicative and may evolve.
The architecture is designed to remain cloud-agnostic, with AWS / Neon as the likely primary database platform.

3.8 Data Architecture Evolution

Stage	Model	Key Characteristics	Trigger
MVP	Single PostgreSQL	• Simplicity • fleet_id isolation	Resource limits
Production	Scaled Relational	• Read replicas • Partitioning	Latency constraints
Scale	Hybrid Model	• Sharding • Tenant isolation • Telemetry separation	Enterprise scale

3.9 Executive Architecture View (Simplified)

```
USERS
  ↓
APPLICATION (Web & Mobile)
  ↓
CORE PLATFORM
  ├──── Rules Engine
  ├──── Performance Engine
  └──── Shift System
  ↓
DATA & REAL-TIME LAYER
  ↓
EXTERNAL SERVICES
  ├──── Telematics (location and sensor data from vehicle)
  ├──── Fleet Fines management
  ├──── Vehicle security services (opening and starting vehicle)
  ├──── Ride-hailing
  ├──── Payments
  └──── Notifications
```

4. Implementation

This document remains at the architectural strategy level.

Guidelines

- PostgreSQL as primary data store
- Event-driven patterns for scale
- Real-time processing as core capability
- Hybrid multi-tenant model at scale
- Compliance enforced across all layers

5. Milestones

1	**MVP Launch (Monolith)**
2	**Production Scaling (Modularisation) – Production Pilot with low volume.**
3	**Service Decomposition**
4	**Enterprise Scale Enablement**

6. Gathering Results (KPIs)

Category	KPI
Operational	• Fleet utilisation (target: near 24/7) • Shift fulfilment rate • Vehicle downtime
Performance	• Telemetry ingestion latency • System response time
Financial	• Payout accuracy • Reconciliation accuracy
Scalability	• Vehicles supported (target: 5,000+) • Stability under load
Compliance	• Audit trail completeness • Enforcement effectiveness